

Foundations of Batch normalization

[Batch normalization]

$$\hat{X}_i = \frac{X_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \hat{X}_i + \beta$$

1.0 Content Block

$$\rho(w_t) - \rho(w^*) \leq (1 - \mu_L)^{2t} (\rho(w_0) - \rho(w^*)) \quad \{\displaystyle \rho(w_t) - \rho(w^*) \leq (1 - \frac{\mu_L}{\rho(w_0) - \rho(w^*)})^{2t} (\rho(w_0) - \rho(w^*))\}$$

2.0 Content Block

$\min_{w \in \mathbb{R}^d} \min_{\gamma \in \mathbb{R}} \text{fOLS}(w, \gamma) = \min_{w \in \mathbb{R}^d} \min_{\gamma \in \mathbb{R}} (2\gamma u^T w + \|w\|_S^2 + \gamma^2)$
 $\{\displaystyle \min_{w \in \mathbb{R}^d} \min_{\gamma \in \mathbb{R}} \text{f}_{\text{OLS}}(w, \gamma) = \min_{w \in \mathbb{R}^d} \min_{\gamma \in \mathbb{R}} (2\gamma \frac{u^T w}{\|w\|_S + \gamma} + \gamma^2)\}$,
 where 0 is excluded to avoid 0 in the denominator. Since the objective is convex with respect to γ , its optimal value could be calculated by setting the partial derivative of the objective against γ to 0. The objective could be further simplified to be

3.0 Content Block

Backpropagation The described BN transform is a differentiable operation, and the gradient of the loss l with respect to the different parameters can be computed directly with the chain rule. Specifically, $\frac{\partial l}{\partial y_i^{(k)}}$ depends on the choice of activation function, and the gradient against other parameters could be expressed as a function of $\frac{\partial l}{\partial y_i^{(k)}}$:

4.0 Content Block

$\tilde{\rho}(w) = w^T B w + w^T A w$, where $B \in \mathbb{R}^d \times \mathbb{R}^d$ is a symmetric matrix and $A \in \mathbb{R}^d \times \mathbb{R}^d$ is a symmetric positive definite matrix. It is proven that the gradient descent convergence rate of the generalized Rayleigh quotient is

5.0 Content Block

$\|\hat{y}_i\|_L^2 \leq \gamma^2 \sigma_j^2 (\|\hat{y}_i\|_L^2 - 1) + \|\hat{y}_i\|_L^2 - 1 + \|\hat{y}_i\|_L^2$, $\|\hat{y}_i\|_L^2 - 1 + \|\hat{y}_i\|_L^2$, $\|\hat{y}_i\|_L^2 - 1 + \|\hat{y}_i\|_L^2$ (Since the gradient magnitude represents the Lipschitzness of the loss, this relationship indicates that a batch normalized network could achieve greater Lipschitzness comparatively. Notice that the bound gets tighter when the gradient $\|\hat{y}_i\|_L^2$ correlates with the activation $\hat{y}_i$, which is a common phenomena. The scaling of $\gamma^2 \sigma_j^2$ is also significant, since the variance is often large. Secondly, the quadratic form of the loss Hessian with respect to

activation in the gradient direction can be bounded as

6.0 Content Block

$s_t = s(w_t, \gamma_t) = -\frac{\|w_t\|_3 L_g h(w_t, \gamma_t)}{\|w_t\|_S^3 L_g h(w_t, \gamma_t)}$. For each step, if $h(w_t, \gamma_t) \neq 0$, then update the direction as

7.0 Content Block

Internal covariate shift Each layer in a neural network has inputs that follow a specific distribution, which shifts during training due to two main factors: the random starting values of the network's settings (parameter initialization) and the natural variation in the input data. This shifting pattern affecting the inputs to the network's inner layers is called internal covariate shift. While a strict definition isn't fully agreed upon, experiments show that it involves changes in the means and variances of these inputs during training. Batch normalization was first developed to address internal covariate shift. During training, as the parameters of preceding layers adjust, the distribution of inputs to the current layer changes accordingly, such that the current layer needs to constantly readjust to new distributions. This issue is particularly severe in deep networks, because small changes in shallower hidden layers will be amplified as they propagate within the network, resulting in significant shift in deeper hidden layers. Batch normalization was proposed to reduced these unwanted shifts to speed up training and produce more reliable models. Beyond possibly tackling internal covariate shift, batch normalization offers several additional advantages. It allows the network to use a higher learning rate—a setting that controls how quickly the network learns—without causing problems like vanishing or exploding gradients, where updates become too small or too large. It also appears to have a regularizing effect, improving the network's ability to generalize to new data, reducing the need for dropout, a technique used to prevent overfitting (when a model learns the training data too well and fails on new data). Additionally, networks using batch normalization are less sensitive to the choice of starting settings or learning rates, making them more robust and adaptable.

8.0 Content Block

In artificial neural networks, batch normalization (also known as batch norm) is a normalization technique used to make training faster and more stable by adjusting the inputs to each layer—re-centering them around zero and re-scaling them to a standard size. It was introduced by Sergey Ioffe and Christian Szegedy in 2015. Experts still debate why batch normalization works so well. It was initially thought to tackle internal covariate shift, a problem where parameter initialization and changes in the distribution of the inputs of each layer affect the learning rate of the network. However, newer research suggests it doesn't fix this shift but instead smooths the objective function—a mathematical guide the network follows to improve—enhancing performance. In very deep networks, batch normalization can initially cause a severe gradient explosion—where updates to the network grow uncontrollably large—but this is managed with shortcuts called skip connections in residual networks. Another theory is that batch normalization adjusts data by handling its size and path separately, speeding up training.

9.0 Content Block

$y(k) = BN_{\gamma(k), \beta(k)}(x(k)) = \gamma(k) x(k) - E[x(k)] \sqrt{\frac{\text{Var}[x(k)] + \epsilon}{\beta(k)}}$, where $y(k)$ is passed on to next layers instead of $x(k)$. Since the parameters are fixed in this transformation, the batch normalization procedure is essentially applying a linear transform to the activation function.